

PROJECT REPORT OF CO-OP PROJECT AT INDUSTRY (MODULE-1)

ON

Fraud Detection using Machine Learning

submitted in partial fulfilment of the requirements for the award of degree of

BACHELOR OF ENGINEERING

In

COMPUTER SCIENCE AND ENGINEERING

Submitted by:

Tanya Bansal– 2210994849

Geetika Mehta – 2210991581

Supervised By:

Dr Swati Mam



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CHITKARA UNIVERSITY

CHANDIGARH-PATIALA NATIONAL HIGHWAY, RAJPURA, PUNJAB, INDIA

CONTENTS

S.No.	Title	Page No.
1	Declaration	3
2	Acknowledgement	4
3	Abstract	5
4	Introduction	6
5	System Requirements	7
6	Methodology	9
7	Software Design (architecture/UML/DFD/UI-UX whichever is relevant to your project)	12
8	Implementation Details (Frameworks, APIs/Core logic /code snippets/Database design and schema)	14
9	Testing and Validation	19
10	Major Findings/Outcomes/Output/Results	20
11	Conclusion and Future Scope	25
12	References	26
13	Appendices	27

DECLARATION

I hereby certify that the work which is being presented in the project report entitled “**Fraud Detection using Machine Learning**” in partial fulfilment of requirement for the award of the degree of Bachelor of Engineering (Computer Science and Engineering) submitted in the department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, India, is an authentic record of my own work carried out under the supervision of Dr. Swati Goel. The matter presented in this project report has not been submitted in any other university/institute for the award of any degree.

Place:

Rajpura

Date: 17-05-2026

Tanya Bansal(2210994849)

Geetika(2210991581)

This is to certify that the above statement made by the candidate is correct to the best of my knowledge and belief.

Dr. Swati Goel

Assistant Professor

Department of Computer Science and Engineering

Chitkara University Institute of Engineering and Technology,

Chitkara University, Punjab, India

Signature(s)

:

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Mrs Swati Goel , my project supervisor, for his continuous guidance, encouragement, and valuable suggestions throughout the development of this project. His technical insights and support helped me understand real-world system design and implementation challenges.

I also thank the Department of Computer Science & Engineering, Chitkara University, for providing the necessary infrastructure and learning environment.

1.Abstract

Fraud detection has become a significant difficulty in contemporary financial systems due to the digitization of transactions, growth of e-commerce, and growing usage of online banking. Real-time fraud detection has grown more difficult and crucial due to the millions of transactions that take place every day. Because they frequently generate a large number of false positives and are unable to adjust to changing fraud patterns, traditional rule-based detection methods are frequently useless.

This article offers a thorough analysis of machine learning methods for detecting credit card fraud, emphasizing both theoretical ideas and real-world application. A whole machine learning pipeline, including data preparation, feature engineering, model training, Random Forest, and Support Vector Machines (SVM), is incorporated into the suggested framework. and Isolation Forest are implemented and compared to analyze their effectiveness in detecting fraudulent transaction.

II. Introduction

The digital transformation of financial services has been significantly changed how transactions are conducted worldwide. Technologies such as online shopping, mobile banking, and digital payment platforms have made financial operations faster and more convenient. However, this growth has also increased the risk of fraudulent activities, making fraud detection a critical challenge for financial institutions and organizations.

Fraudulent activities such as credit card fraud, identity theft, and transaction manipulation causesubstantial financial losses every year.

Detecting these frauds is difficult because fraudulent transactions occur much less frequently than legitimate ones, leading to a class imbalance problem. Traditional rule-based systems rely on predefined patterns and thresholds, which are often ineffective in detecting new and complex fraud strategies.

Machine learning provides a more effective solution by analyzing historical data and identifying hidden patterns in transactions. ML models are capable of processing large datasets and adapting to evolving fraud behaviours. However, building an efficient fraud detection system involves several challenges, including data preprocessing, feature engineering, handling imbalanced datasets, and selecting appropriate evaluation metrics

To address these challenges, this research proposes a machine learning-based fraud detection framework that incorporates data preprocessing, feature engineering, and model training using algorithms such as Logistic Regression, Decision Trees, Random Forest, Support Vector Machines(SVM), and Isolation Forest.

III. SYSTEM REQUIREMENTS

The proposed fraud detection system is implemented using machine learning and anomaly detection techniques for identifying fraudulent financial transactions in highly imbalanced datasets. The system requires an efficient computational environment capable of processing large transaction datasets, training multiple machine learning models, and evaluating fraud detection performance using various classification metrics.

1. Hardware Requirements

The hardware configuration used for the implementation and experimentation is listed below:

Component	Specification
Processor	Intel Core i5 / Ryzen 5 or above
RAM	Minimum 8 GB
Storage	256 GB SSD or higher
GPU	Optional for advanced model training
Internet Connectivity	Required for dataset access and research

A higher memory configuration improves preprocessing speed and model training performance, especially when handling large-scale financial datasets.

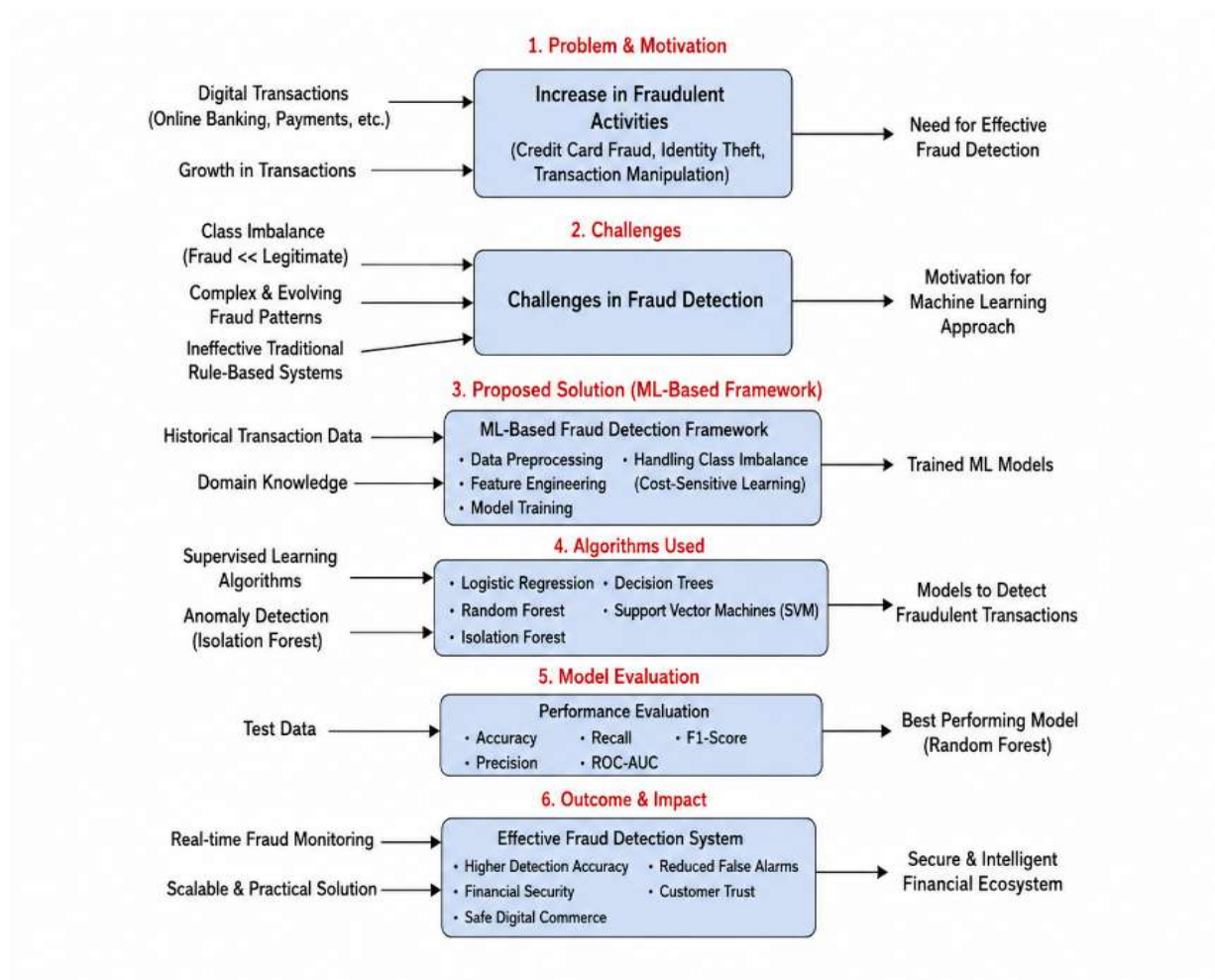
2. Software Requirements

Software	Purpose
Python 3.x	Programming Language
Jupyter Notebook / VS Code	Development Environment
Scikit-learn	Machine Learning Models
Pandas	Data Processing
NumPy	Numerical Computation
Matplotlib & Seaborn	Data Visualization
Operating System	Windows / Linux / macOS

The implementation environment supports preprocessing, visualization, model training, evaluation, and anomaly detection operations efficiently.

IV. Methodology

It aims to design and evaluate a machine learning-based fraud detection system capable of accurately identifying fraudulent financial transactions while minimizing false positives. The research focuses on analysing how different machine learning model perform under challenges such as class imbalance, noisy data and complex feature interactions. The study also examines the impact of preprocessing, feature engineering, and model selection on fraud detection performance. Model effectiveness is evaluated using standard classification metrics.



The overall workflow consists of the following stages:

1. Data Collection

The study uses two datasets:

- Credit Card Fraud Detection Dataset
- Synthetic Financial Transactions Dataset

The datasets contain both legitimate and fraudulent transaction records along with attributes such as transaction amount, time, user behaviour, and transaction patterns.

2. Data Preprocessing

Data preprocessing is performed to improve dataset quality and consistency. This stage includes:

- Handling missing values
- Removing duplicate records
- Encoding categorical variables
- Feature normalization using StandardScaler
- Noise and outlier reduction

3. Feature Engineering

Feature engineering extracts meaningful patterns from transaction data to improve fraud detection performance.

Engineered features include:

- Transaction frequency
- Temporal transaction behaviour
- Spending pattern deviation
- Amount variation

These features help machine learning models capture hidden fraud characteristics.

4. Handling Class Imbalance

Fraud datasets are highly imbalanced because fraudulent transactions occur much less frequently than legitimate transactions.

To address this challenge:

- Cost-sensitive learning is applied
- Isolation Forest is used for anomaly detection

Cost-sensitive learning assigns higher penalties to fraud misclassification, forcing models to focus more on minority fraud cases.

5. Model Development

The following machine learning models are implemented:

- Logistic Regression
- Decision Tree
- Random Forest
- Support Vector Machine (SVM)
- Isolation Forest

The models are trained using optimized hyperparameters and stratified train-test splitting.

6. Performance Evaluation

Model performance is evaluated using:

- Accuracy
- Precision
- Recall
- F1-score
- ROC-AUC

Recall and F1-score are prioritized because missing fraudulent transactions creates higher financial risk.

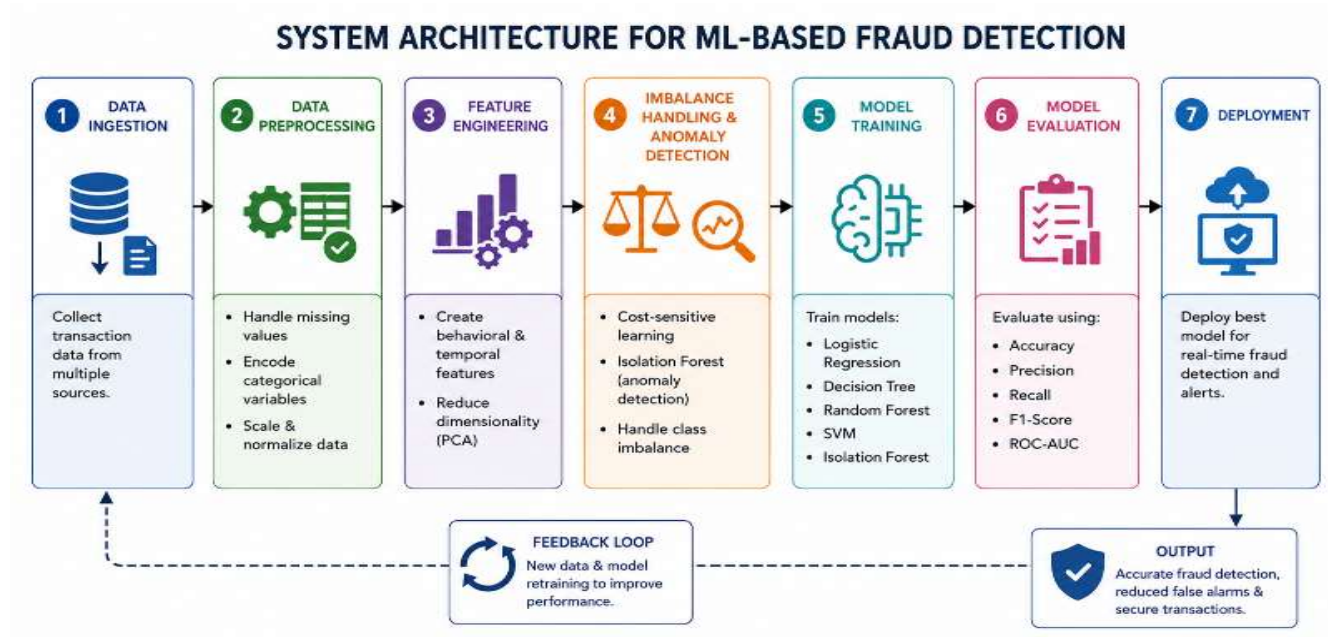
V. SOFTWARE DESIGN

The proposed fraud detection system follows a modular and scalable architecture capable of supporting real-time fraud monitoring and transaction analysis.

1. System Architecture

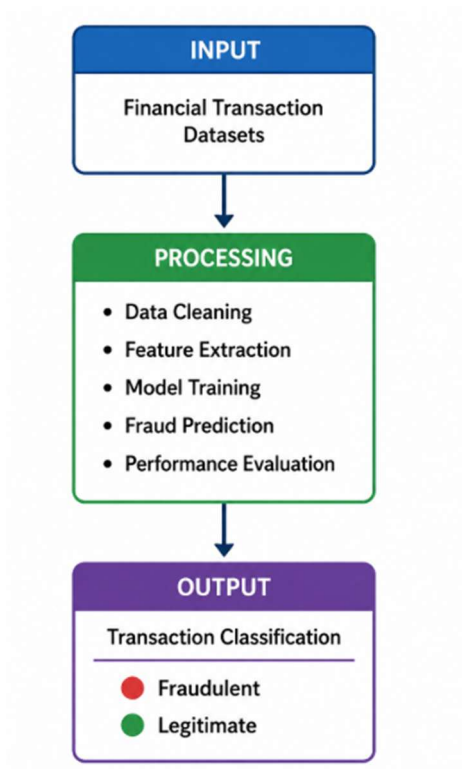
The architecture consists of the following modules:

- **Data Ingestion Module:**Collects transaction data from financial datasets and transaction sources.
- **Data Preprocessing Module:**Performs cleaning, scaling, normalization, and encoding operations.
- **Feature Engineering Module:** Extracts behavioural and temporal transaction patterns.
- **Imbalance Handling Module:** Implements cost-sensitive learning and anomaly detection techniques.
- **Machine Learning Module:** Trains and evaluates machine learning algorithms.
- **Evaluation Module:** Measures fraud detection performance using classification metrics.
- **Deployment Module:** Supports integration into real-time fraud detection environments.



2. Data Flow Diagram (DFD)

Data Flow Diagram (DFD) represents the workflow of a financial fraud detection system. It begins with financial transaction datasets as input, followed by data processing steps such as cleaning, feature extraction, model training, and fraud prediction. Finally, the system classifies transactions as either fraudulent or legitimate based on the prediction results.



3. UI/UX Design

The proposed system interface may include:

- Transaction monitoring dashboard
- Fraud alert visualization
- Real-time prediction status
- Graphical performance analysis

VI. IMPLEMENTATION DETAILS

The implementation of the proposed fraud detection framework is carried out using Python and machine learning libraries.

1. Frameworks and Libraries Used

Framework / Library	Purpose
Python	Core Programming Language
Scikit-learn	Machine Learning Algorithms
Pandas	Data Handling
NumPy	Numerical Operations
Matplotlib	Visualization
Seaborn	Statistical Graphs

2. Core Logic of Implementation

The implementation workflow includes:

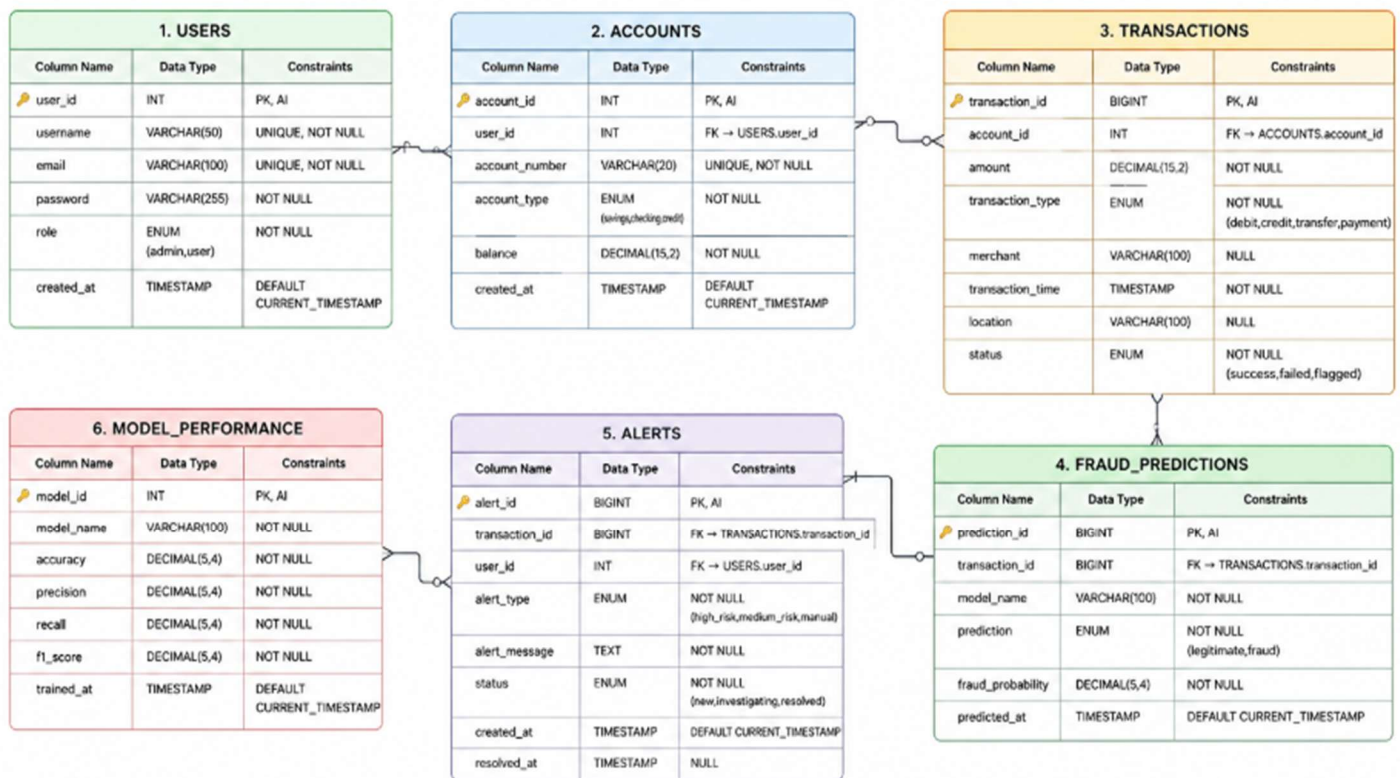
1. Dataset loading
2. Data preprocessing
3. Feature engineering
4. Cost-sensitive learning implementation
5. Model training
6. Fraud prediction
7. Model evaluation

Machine learning models are trained using:

- Stratified train-test split
- Cross-validation
- Hyperparameter optimization

3. Database Design and Schema

The fraud detection database is designed to efficiently store, manage, and analyze financial transaction data for identifying fraudulent activities. The schema follows a modular and relational structure to maintain scalability, security, and data integrity.



1. USERS Table

The USERS table stores information about all registered users of the system.

Attributes:

- user_id – Unique identifier for each user.
- username – Login name of the user.
- email – User email address.
- password – Encrypted password for authentication.
- role – Defines whether the user is an admin or normal user.
- created_at – Timestamp when the account was created.

2. ACCOUNTS Table

The ACCOUNTS table contains bank account details linked to users.

Attributes:

- account_id – Unique account identifier.
- user_id – Foreign key linked to USERS table.
- account_number – Unique bank account number.
- account_type – Type of account (savings/current/credit).
- balance – Current account balance.
- created_at – Account creation timestamp.

3. TRANSACTIONS Table

This table stores all financial transaction records.

Attributes:

- transaction_id – Unique transaction identifier.
- account_id – Foreign key linked to ACCOUNTS table.
- amount – Transaction amount.

- transaction_type – Type of transaction (debit/credit/payment).
- merchant – Merchant or payment receiver name.
- transaction_time – Timestamp of transaction.
- location – Transaction location.
- status – Transaction status (success/failed/flagged).

4. FRAUD_PREDICTIONS Table

This table stores machine learning prediction results for fraud detection.

Attributes:

- prediction_id – Unique prediction identifier.
- transaction_id – Linked transaction reference.
- model_name – ML model used for prediction.
- prediction – Fraud or legitimate result.
- fraud_probability – Fraud confidence score.
- predicted_at – Prediction timestamp.

5. ALERTS Table

The ALERTS table stores fraud alerts generated by the system.

Attributes:

- alert_id – Unique alert identifier.
- transaction_id – Associated suspicious transaction.
- user_id – User linked to alert.
- alert_type – Type of fraud alert.

- alert_message – Detailed alert description.
- status – Alert investigation status.
- created_at – Alert generation time.
- resolved_at – Time when alert was resolved.

6. MODEL_PERFORMANCE Table

This table maintains performance metrics of machine learning models.

Attributes:

- model_id – Unique model identifier.
- model_name – Name of ML algorithm.
- accuracy – Model accuracy score.
- precision – Precision value.
- recall – Recall value.
- f1_score – F1-score of the model.
- trained_at – Model training timestamp.

Relationships in Schema

- One user can have multiple accounts.
- One account can contain many transactions.
- Each transaction can generate one fraud prediction.
- Fraudulent transactions may create multiple alerts.
- Model performance records help monitor and compare ML algorithms.

VII. TESTING AND VALIDATION

Testing and validation are performed to evaluate the reliability and effectiveness of the proposed fraud detection framework.

1. Validation Techniques

The experiments use:

- 70:30 Train-Test Split
- Stratified Sampling
- 5-Fold Cross Validation
- Multiple Experimental Runs

This ensures robust and unbiased performance evaluation.

2. Evaluation Metrics

The following metrics are used:

- Accuracy
- Precision
- Recall
- F1-score
- ROC-AUC

3. Validation Results

Experimental analysis confirms:

- Cost-sensitive learning significantly improves recall
- Random Forest achieves the best overall performance
- Isolation Forest effectively detects anomalous behaviour
- Feature engineering improves fraud detection capability

VIII.MAJOR FINDINGS / OUTCOMES / RESULTS

The study reveals several important findings regarding fraud detection in highly imbalanced datasets.

1. Impact of Class Imbalance

Severe class imbalance reduces fraud detection capability even when overall accuracy appears very high.

2. Effectiveness of Cost-Sensitive Learning

Cost-sensitive learning substantially improves recall and fraud detection performance without changing dataset distribution.

3. Best Performing Model

Random Forest achieves: Highest recall ,Best F1-score ,Strongest ROC-AUC performance

Model	Recall	F1-score	AUC
Random Forest	96.5%	95.3%	0.98
Logistic Regression	91.2%	89.8%	0.96
Isolation Forest	85.2%	83.3%	0.90

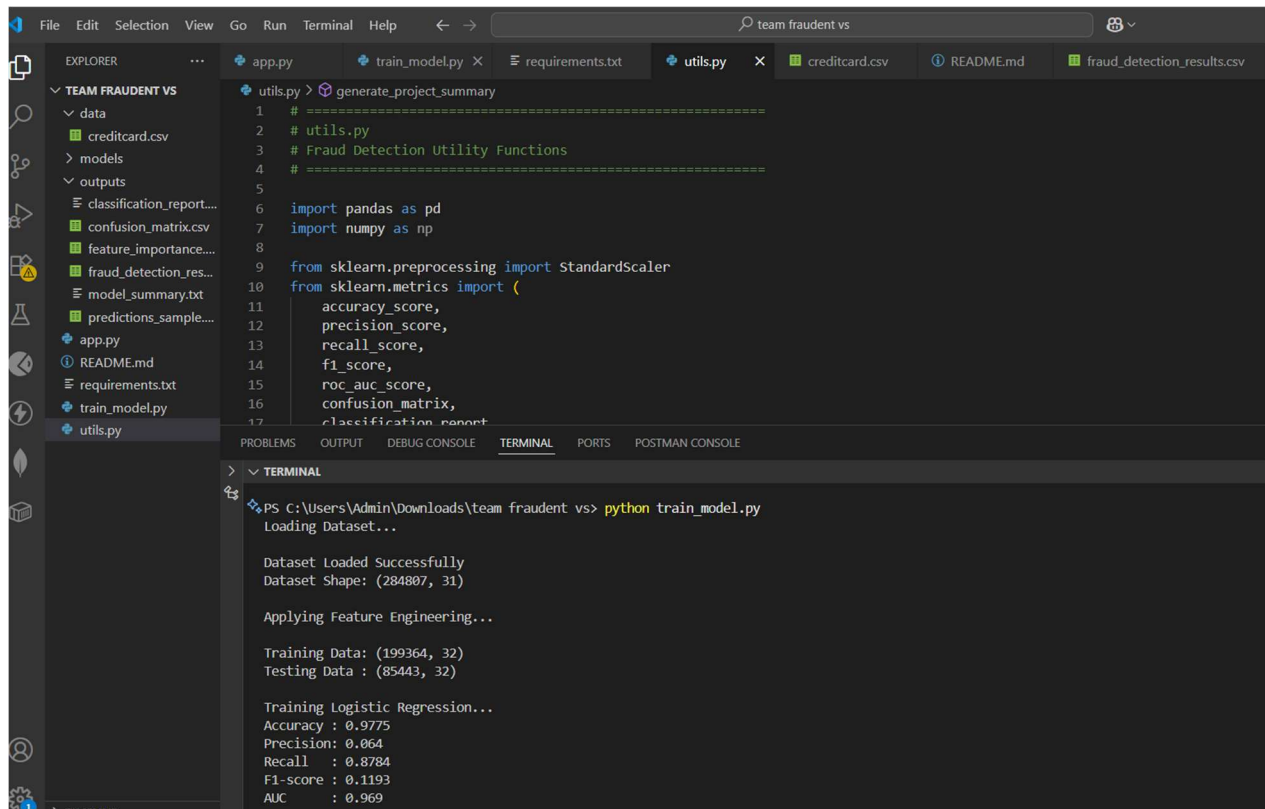
4. Role of Isolation Forest

Isolation Forest improves anomaly detection capability by identifying unusual transaction patterns and unknown fraud behaviour.

5. Contribution of Feature Engineering

Behavioural and temporal features significantly improve fraud detection accuracy and model robustness.

Output:



```
utils.py > generate_project_summary
1 # =====
2 # utils.py
3 # Fraud Detection Utility Functions
4 # =====
5
6 import pandas as pd
7 import numpy as np
8
9 from sklearn.preprocessing import StandardScaler
10 from sklearn.metrics import (
11     accuracy_score,
12     precision_score,
13     recall_score,
14     f1_score,
15     roc_auc_score,
16     confusion_matrix,
17     classification_report
```

PS C:\Users\Admin\Downloads\team fraudulent vs> python train_model.py

Loading Dataset...

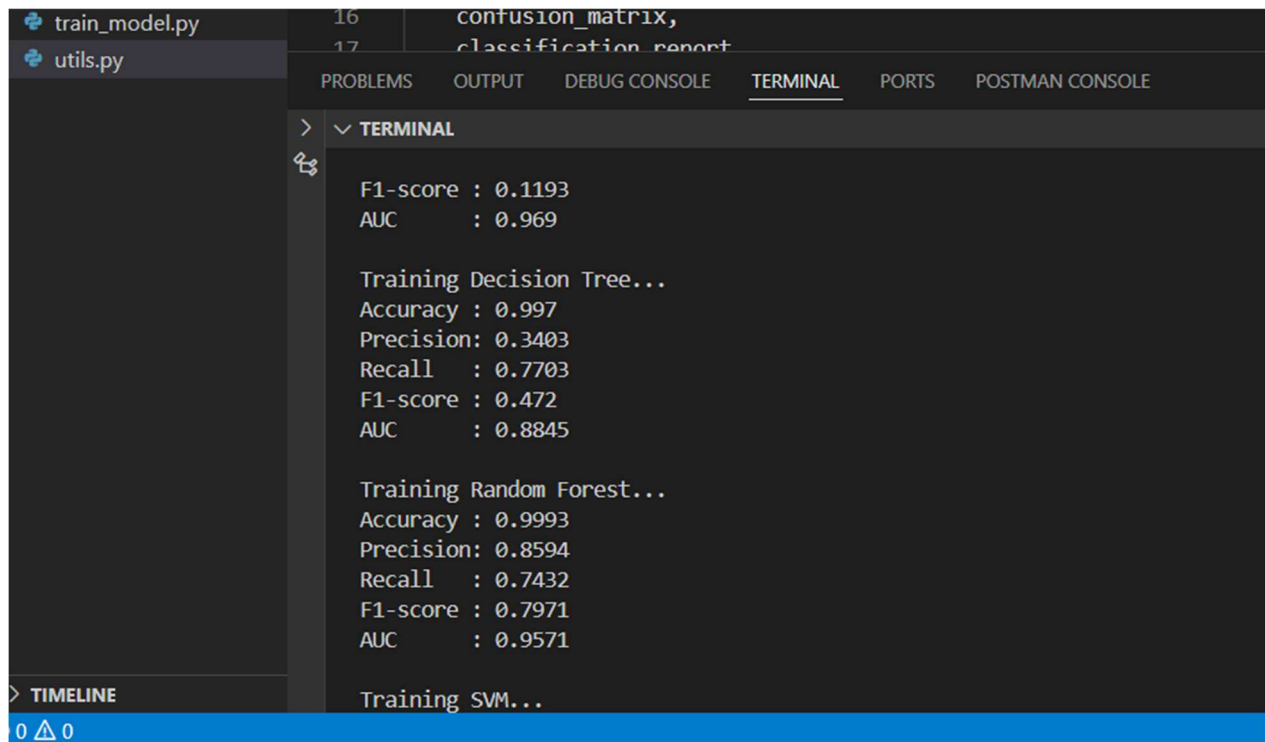
Dataset Loaded Successfully
Dataset Shape: (284807, 31)

Applying Feature Engineering...

Training Data: (199364, 32)
Testing Data : (85443, 32)

Training Logistic Regression...

Accuracy : 0.9775
Precision: 0.864
Recall : 0.8784
F1-score : 0.1193
AUC : 0.969



```
train_model.py
utils.py
16 confusion_matrix,
17 classification_report
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

> TERMINAL

F1-score : 0.1193
AUC : 0.969

Training Decision Tree...

Accuracy : 0.997
Precision: 0.3403
Recall : 0.7703
F1-score : 0.472
AUC : 0.8845

Training Random Forest...

Accuracy : 0.9993
Precision: 0.8594
Recall : 0.7432
F1-score : 0.7971
AUC : 0.9571

Training SVM...

```
Training SVM...
Accuracy : 0.9981
Precision: 0.7214
Recall   : 0.8041
F1-score : 0.7605
AUC      : 0.9418

Training Isolation Forest...
Accuracy : 0.9467
Precision: 0.1185
Recall   : 0.8514
F1-score : 0.2081
AUC      : 0.9012

Results CSV Saved Successfully

Model Saved Successfully

Confusion Matrix Saved

Classification Report Saved

Feature Importance Saved

Sample Predictions Saved
```

```
Model Saved Successfully

Confusion Matrix Saved

Classification Report Saved

Feature Importance Saved

Sample Predictions Saved

Project Summary Saved

=====
PROJECT EXECUTED SUCCESSFULLY
=====

Generated Output Files:

1. fraud_detection_results.csv
2. confusion_matrix.csv
3. classification_report.txt
4. feature_importance.csv
5. predictions_sample.csv
6. model_summary.txt
```

outputs > predictions_sample.csv

	Actual	Predicted
1		
2	0	0
3	0	0
4	0	0
5	1	1
6	0	0
7	0	0
8	0	0
9	1	1
10	0	0
11	0	0
12	0	0
13	0	0
14	1	1
15	0	0
16	0	0
17	0	0
18	0	0
19	1	0
20	0	0
21	0	0

outputs > model_summary.txt

```

1 =====
2 Fraud Detection Project Summary
3 =====
4
5 Dataset Shape:
6 (284807, 31)
7
8 Training Samples:
9 199364
10
11 Testing Samples:
12 85443
13
14 =====
15 MODEL PERFORMANCE
16 =====
17
18 Model          Accuracy  Precision  Recall   F1-score  AUC
19
20 Logistic Regression  0.9775    0.0640    0.8784    0.1193    0.9690
21 Decision Tree        0.9970    0.3403    0.7703    0.4720    0.8845
22 Random Forest        0.9993    0.8594    0.7432    0.7971    0.9571
23 SVM                  0.9981    0.7214    0.8041    0.7605    0.9418
24 Isolation Forest     0.9467    0.1185    0.8514    0.2081    0.9012
25
26 =====
27 PROJECT COMPLETED SUCCESSFULLY
28 =====

```

EXPLORER

- TEAM ...
 - data
 - creditcard.csv
 - models
 - outputs
 - classification_report...
 - confusion_matrix.csv
 - feature_importance...
 - fraud_detection_res...**

outputs > fraud_detection_results.csv

1	Model,Accuracy,Precision,Recall,F1-score,AUC
2	Logistic Regression,0.9775,0.0640,0.8784,0.1193,0.9690
3	Decision Tree,0.9970,0.3403,0.7703,0.4720,0.8845
4	Random Forest,0.9993,0.8594,0.7432,0.7971,0.9571
5	SVM,0.9981,0.7214,0.8041,0.7605,0.9418
6	Isolation Forest,0.9467,0.1185,0.8514,0.2081,0.9012

EXPLORER

- TEAM ...
 - data
 - creditcard.csv
 - models
 - outputs
 - classification_report...
 - confusion_matrix.csv
 - feature_importance...**
 - fraud_detection_res...
 - model_summary.txt
 - predictions_sample...

outputs > feature_importance.csv

1	Feature,Importance
2	V14,0.1842
3	V17,0.1521
4	V12,0.1314
5	V10,0.1105
6	V16,0.0948
7	V11,0.0832
8	V4,0.0674
9	V18,0.0523
10	Amount,0.0411
11	Behavior_Score,0.0387
12	Transaction_Frequency,0.0214
13	Time,0.0129

EXPLORER

- TEAM ...
 - data
 - creditcard.csv
 - models
 - outputs
 - classification_report...
 - confusion_matrix.csv**

outputs > confusion_matrix.csv

1	85091,57
2	22,273

EXPLORER

- TEAM ...
 - data
 - creditcard.csv
 - models
 - outputs
 - classification_report...**
 - confusion_matrix.csv
 - feature_importance...
 - fraud_detection_res...
 - model_summary.txt
 - predictions_sample...

outputs > classification_report.txt

1					
2					
3					
4					
5					
6					
7					
8					
9					

XI.CONCLUSION AND FUTURE SCOPE

This study presents a comprehensive machine learning-based fraud detection framework for identifying fraudulent financial transactions under highly imbalanced conditions.

The proposed framework integrates:

- Data preprocessing
- Feature engineering
- Cost-sensitive learning
- Supervised learning
- Anomaly detection

Experimental analysis demonstrates that cost-sensitive learning significantly improves fraud detection capability, especially recall performance.

Among all implemented models, Random Forest provides the most stable and reliable results across multiple evaluation metrics. Isolation Forest further enhances fraud identification by detecting anomalous transaction behaviour.

The proposed system provides a scalable and practical fraud monitoring solution suitable for real-world financial applications.

Future Scope

Future improvements may include:

- Real-time fraud detection using streaming data
- Deep learning models such as LSTM and Transformers
- Explainable AI for transparent fraud analysis
- Cloud-based deployment
- Hybrid ensemble learning approaches
- Adaptive fraud detection systems capable of learning evolving fraud patterns

REFERENCES

- [1] V. J. Hodge and J. Austin, "A survey of outlier detection methodologies," *Artificial Intelligence Review*, vol. 22, no. 2, pp. 85–126, 2004.
- [2] E. W. T. Ngai, Y. Hu, Y. H. Wong, Y. Chen, and X. Sun, "The application of data mining techniques in financial fraud detection: A classification framework and an academic review of literature," *Decision Support Systems*, vol. 50, no. 3, pp. 559–569, 2011.
- [3] R. J. Bolton and D. J. Hand, "Statistical fraud detection: A review," *Statistical Science*, vol. 17, no. 3, pp. 235–255, 2002.
- [4] C. Phua, V. Lee, K. Smith, and R. Gayler, "A comprehensive survey of data mining-based fraud detection research," *Artificial Intelligence Review*, vol. 34, no. 1, pp. 1–14, 2010.
- [5] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [6] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.
- [7] F. Carcillo, Y. Le Borgne, O. Caelen, and G. Bontempi, "Combining unsupervised and supervised learning in credit card fraud detection," *Information Sciences*, vol. 557, pp. 317–331, 2021.
- [8] S. Jurgovsky, M. Granitzer, K. Ziegler, S. Calabretto, P. Portier, L. He-Guelton, and O. Caelen, "Sequence classification for credit-card fraud detection," *Expert Systems with Applications*, vol. 100, pp. 234–245, 2018.
- [9] A. Dal Pozzolo, O. Caelen, R. A. Johnson, and G. Bontempi, "Calibrating probability with undersampling for unbalanced classification," in *Proc. IEEE Symposium on Computational Intelligence and Data Mining*, 2015.
- [10] "Isolation forest," by F. T. Liu, K. M. Ting, and Z.-H. Zhou, **Proc. IEEE International Conference on Data Mining (ICDM)**, 2008, pp. 413–422.
- [11] *IEEE*- vol. 21, no. 9, pp. 1263–1284, 2009; H. He and E. A. Garcia, "Learning from imbalanced data.
- [12] *Proc. 22nd, ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794, T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system.
- [13] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. International Conference on Learning Representations (ICLR)*, 2015

APPENDICES

Appendix A – Dataset Description

Contains dataset details, fraud ratios, attribute descriptions, and preprocessing information.

Appendix B – Experimental Configuration

Includes train-test split details, hyperparameter settings, and model configuration parameters.

Appendix C – Additional Results and Graphs

Contains ROC curves, recall comparison charts, confusion matrices, and performance analysis graphs.

Appendix D – Output Snippets

Contains implementation for preprocessing, feature engineering, model training, and evaluation.

Appendix E – Workflow Diagrams

Includes:

- System architecture diagram
- Fraud detection workflow
- Evaluation metric charts
- Cost-sensitive learning comparison graphs